

מדריך הגנה על הפרויקט – LoRa

Defibrillator Network Simulator

מדריך אקדמי מקיף להגנת פרויקט גמר

מטרת מסמך זה: להכין את הסטודנט להגנה קפדנית על הפרויקט.
כל שאלה, כל שורת קוד, וכל החלטה ארכיטקטורלית מוסברת כאן לעומק.

פרק 1: ארכיטקטורה ו-Tech Stack

1.1 למה שני שרתים?

זוהי אחת השאלות הראשונות שתישאל. התשובה המלאה:

הדרישה: "שימוש לפחות בשני שרתים אחד מהם Express למימוש JWT עם יכולת רענון"

ההגיון הארכיטקטורלי (Microservices):

הפרדת שרת האימות (Auth Server) מהאפליקציה הראשית היא pattern ידוע בתעשייה.

Client (Browser)

|

:3000

|

Next.js Web App
(Vercel)

- UI דפיי
- API Routes
- Map simulation
- CMS pages

|

:4000

|

Express Auth Server
(Render / Railway)

- POST /auth/login
- POST /auth/refresh
- POST /auth/logout
- GET /auth/verify

PostgreSQL + MongoDB

יתרונות ההפרדה:

1. **Security Isolation**: אם ה-Next.js נפרץ, קוד האימות בשרת נפרד
2. **Scalability**: כל שרת יכול לשנות scale בנפרד
3. **Separation of Concerns**: כל שרת אחראי לתחום אחד בלבד
4. **Technology Freedom**: Express מתאים יותר ל-JWT middleware

Next.js – App Router 1.2

מה זה Next.js?

Next.js הוא Framework של React שמוסיף:

- **SSR (Server-Side Rendering)**: HTML נוצר בשרת, לא בדפדפן – טוב ל-SEO
- **API Routes**: קוד Backend ישר ב-Next.js ללא שרת נפרד
- **File-based Routing**: `src/app/simulator/page.tsx` = `/simulator` URL אוטומטית

App Router לעומת Pages Router (Next.js 13+)

App Router (חדש)	Pages Router (ישן)
<code>src/app/page.tsx</code>	<code>pages/index.tsx</code>
כל דף = Server Component כברירת מחדל	כל דף = Client Component
async component function	<code>getServerSideProps</code>
<code>app/api/*/route.ts</code>	<code>pages/api/*.ts</code>

Server Components לעומת Client Components

```
// server component (ברירת מחדל) – בשרת בלבד
// window/DOM-ישרות, אין גישה ל DB queries יכול לעשות
export default async function Page() {
  const data = await fetch('...');
```

```

    return <div>{data}</div>;
  }

  // client component – רץ בדפדפן
  'use client'; // הסמן הכרחי!
  import { useState } from 'react'; // hooks עובדים רק כאן
  export default function InteractivePage() {
    const [count, setCount] = useState(0);
    return <button onClick={() => setCount(c => c+1)}>{count}</button>;
  }

```

API Routes ב-App Router:

```

src/app/api/emergency/trigger/route.ts
└─ route.ts = endpoint מיוחד
    export async function POST(req)
    export async function GET()
    מפה ל: POST /api/emergency/trigger
    GET /api/emergency/trigger

```

PostgreSQL vs MongoDB 1.3 – למה שתיים?

זוהי השאלה הקריטית ביותר בהגנה!

הדרישה: "מימוש שתי סוגי דאטה בייס (NoSQL, SQL)"

PostgreSQL (SQL, Relational) – לנתונים מובנים:

```

-- שורות קשיחות, עמודות מוגדרות מראש – volunteers טבלת
CREATE TABLE volunteers (
  id SERIAL PRIMARY KEY,
  first_name VARCHAR(100) NOT NULL,
  mobile_number VARCHAR(20) UNIQUE NOT NULL,
  has_lora BOOLEAN DEFAULT FALSE,
  has_defibrillator BOOLEAN DEFAULT TRUE
);

-- מורכב JOIN ניתן לבצע:
SELECT v.first_name, vs.total_points, vs.badges
FROM volunteers v

```

```
LEFT JOIN volunteer_scores vs ON v.id = vs.volunteer_id
WHERE v.has_lora = TRUE;
```

מתאים ל-PostgreSQL כי:

- נתוני מתנדבים = סכמה קבועה (שם, טלפון, תעודה)
- נדרשים JOINS בין טבלאות (volunteers + scores)
- ACID transactions (עדכון ניקוד – הכל או כלום)
- Unique constraints (מספר טלפון = ייחודי)

MongoDB (NoSQL, Document) – לנתונים דינמיים/גיאוגרפיים:

```
{
  "_id": "ObjectId('...')",
  "volunteerId": 42,
  "location": {
    "type": "Point",
    "coordinates": [34.9555, 31.9439]
  },
  "batteryLevel": 87,
  "signalStrength": -75,
  "meshHops": 2,
  "lastTransmission": "2026-07-01T15:30:00Z",
  "isOnline": true
}
```

מתאים ל-MongoDB כי:

- GeoJSON support מובנה עם `2dsphere` index
- שאילתת `$geoWithin / $centerSphere` – 1 שורה במקום Haversine ב-SQL
- אירועי חירום = documents עם arrays מקוננים
- נתוני telemetry = schema גמיש

ההבדל הקריטי – שאילתת גיאוגרפית:

```
// MongoDB – שורה אחת בלבד!
db.device_telemetry.find({
  location: { $geoWithin: { $centerSphere: [[34.95, 31.97], 0.00313] } }
});

// PostgreSQL – מורכבת Haversine formula דרושה:
```

```
SELECT * FROM locations WHERE 2 * 6371 * ASIN(SQRT(
  POWER(SIN(RADIANS(lat - 31.97) / 2), 2) +
  COS(RADIANS(31.97)) * COS(RADIANS(lat)) *
  POWER(SIN(RADIANS(lng - 34.95) / 2), 2))) <= 20;
```

פרק 2: טכנולוגיית LoRa ו-Meshtastic

LoRa – Long Range Radio 2.1

מה זה LoRa?

LoRa (Long Range) היא טכנולוגיית רדיו שמשתמשת ב-Chirp Spread Spectrum (CSS):

- מרחק:** עד 15 ק"מ בשטח פתוח
- צריכת חשמל:** מינימלית – סוללת AA יכולה להחזיק שנים
- תדר:** 433MHz בישראל (ISM band – אין צורך ברישיון אישי)
- Bandwidth:** נמוך – מספיק להודעות טקסט ו-GPS coordinates
- Cost:** מכשיר LoRa עולה כ-30-50 דולר

LoRa לעומת Cellular:

פרמטר	LoRa	Cellular
מרחק	עד 15 ק"מ	תלוי תשתית
צריכת חשמל	מינימלית	גבוהה
עלות	חד פעמי	דמי מנוי חודשיים
עצמאות	עצמאי לחלוטין	תלוי ספק
כיסוי	בכל מקום	רק איפה יש אנטנות

Meshtastic 2.2 – הרשת החכמה

רשת Mesh לעומת Point-to-Point:

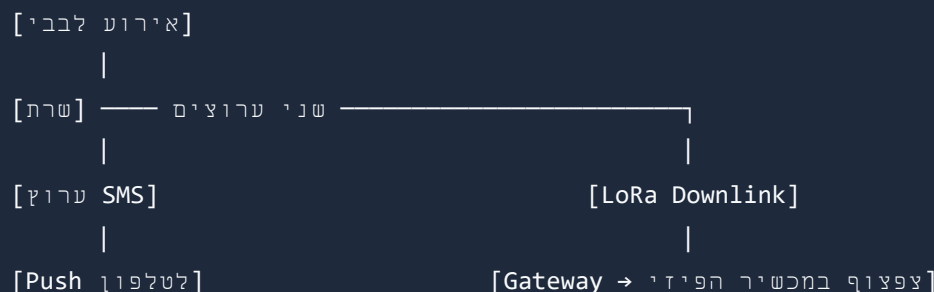
Point-to-Point: [A] —————X [B] (!רחוק מדי)

Mesh Network: [A] → [C] → [D] → [B] (!כל מכשיר ממסר)

ערך Meshtastic:

- **אין תשתית:** הרשת קמה ברגע שמספיק מכשירים בטווח
- **Routing אוטומטי:** Meshtastic מחשב נתיב אופטימלי
- **Encryption: AES-256** – רק חברי הקבוצה קוראים את ההודעות

2.3 הארכיטקטורה ההיברידית



הגאונות: אם מתנדב ללא קליטה סלולרית – ה-LoRa מגיע דרך רשת ה-Mesh!

פרק 3: מבנה הקבצים

```

FinalProject/
├── .gitignore           # מגן על סודות
├── .env.example         # תבנית ללא ערכים אמיתיים
├── README.md
├── DEFENSE_GUIDE.md    # המסמך הזה
├──
├── auth-server/
│   └── src/
│       ├── index.ts    # Express app – נקודת כניסה
│       ├── config.ts   # מרוכזים כאן env vars כל
│       ├── routes/auth.ts # login/refresh/logout/verify
│       ├── middleware/validateToken.ts # בדיקת Access Token
│       └── db/postgres.ts # חיבור PostgreSQL
  
```

```

├── seed-admin.ts    # יוצר admin micha/1234
├── web-app/
│   ├── scripts/seed.ts    # מאכלס PostgreSQL + MongoDB
│   └── src/
│       ├── app/
│       │   ├── layout.tsx    # Root Layout – RTL, Hebrew fonts
│       │   ├── page.tsx      # Homepage – LoRa + Mermaid
│       │   └── api/
│       │       ├── emergency/trigger/route.ts    # הלוגיקה המרכזית!
│       │       ├── volunteers/route.ts
│       │       ├── volunteers/[id]/route.ts
│       │       ├── cms/[slug]/route.ts
│       │       └── gamification/route.ts
│       ├── simulator/page.tsx
│       ├── register/page.tsx
│       ├── info/page.tsx     # מד"א, LoRa, קניית
│       ├── leaderboard/page.tsx
│       └── admin/
│           ├── login/page.tsx
│           └── dashboard/page.tsx
│   ├── components/
│   │   ├── map/EmergencyMap.tsx    # Leaflet – Client only
│   │   ├── MermaidDiagram.tsx
│   │   └── MermaidWrapper.tsx     # dynamic SSR:false
│   └── lib/
│       ├── db/postgres.ts    # Connection Pool
│       ├── db/mongodb.ts     # Singleton Client
│       ├── ors.ts            # OpenRouteService
│       └── mock-data.ts      # demo מייצר 50 מתנדבי

```

פרק 4: זרימות קריטיות

4.1 מה קורה כשלווחצים "הפעל התראת חירום"?

[1] לחיצה על המפה

```
EmergencyMap: onMapClick(e.latlng.lat, e.latlng.lng)
```

[2] simulator/page.tsx – React state update

```
setEmergencyLocation({ lat, lng })
```

```

[3] לחיצה על כפתור → fetch POST /api/emergency/trigger

[4] Next.js API Route → route.ts

[5] MongoDB $geoWithin שאילתה:
center = [lng, lat], radius = radiusKm / 6378.1 radians
בטווח device_telemetry מחזיר: מסמכי

[6] PostgreSQL JOIN:
SELECT v.*, vs.total_points FROM volunteers v
LEFT JOIN volunteer_scores vs ON v.id = vs.volunteer_id
WHERE v.id = ANY([...ids...])

[7] OpenRouteService API (לכל מתנדב):
/directions/cycling-regular/geojson
מחזיר: distanceKm, durationMin, GeoJSON LineString

[8] MongoDB: insertOne ל-emergency_events

[9] PostgreSQL: UPDATE volunteer_scores (ניקוד + badges)

[10] JSON Response ← { notifiedVolunteers, totalNotified, ... }

[11] React: setVolunteers(data.notifiedVolunteers)

[12] EmergencyMap.tsx:
useEffect → clearLayers → L.marker + L.polyline + fitBounds

```

זמן כולל: כ-5-12 שניות (תלוי ב-ORS response time)

4.2 מחזור חיי JWT

LOGIN:

```

Browser → POST /auth/login {username, password}
  ← Auth Server: bcrypt.compare → jwt.sign(accessToken, 15m)
  uidv4() → jti → jwt.sign(refreshToken, 7d)
  INSERT INTO refresh_tokens (jti, expires_at)
  ← { accessToken } + Set-Cookie: refreshToken (HttpOnly!)

```

REQUEST:

```

Browser → GET /api/... Authorization: Bearer {accessToken}
  → Next.js → GET :4000/auth/verify → jwt.verify(token)
  ← { valid: true }

```


REFRESH (access token פג תוקן):

Browser → POST /auth/refresh (cookie אנשלח אוטומטית)

→ Auth Server:

1. `jwt.verify(refreshToken)`
2. `SELECT WHERE jti=$1 AND revoked=FALSE AND expires_at > NOW()`
3. `UPDATE SET revoked=TRUE` (ביטול הישן – ROTATION!)
4. `newJti = uuidv4() → jwt.sign(new tokens)`
5. `INSERT INTO refresh_tokens (newJti, ...)`

← { newAccessToken } + Set-Cookie: new refreshToken

LOGOUT:

Browser → POST /auth/logout

- `UPDATE refresh_tokens SET revoked=TRUE WHERE jti=$1`
 → `res.clearCookie('refreshToken')`

4.3 הגנות אבטחה שמומשו

הגנה	מגן מ...	איפה בקוד
HttpOnly Cookie	XSS – JavaScript גונב refresh token לא	<code>auth.ts: res.cookie(..., {httpOnly: true})</code>
SameSite=Strict	CSRF attacks	<code>auth.ts: sameSite: 'strict'</code>
Refresh Token Rotation	Replay attacks	לפני <code>auth.ts: revoked=TRUE</code> הנפקת חדש
Access Token קצר (15m)	גניבת token = נזק מוגבל	<code>config.ts: accessTokenExpiry: '15m'</code>
Parameterized Queries	SQL Injection	<code>pool.query('... WHERE id=\$1', [id])</code>
bcrypt	Rainbow Table attacks	<code>bcrypt.compare(password, hash)</code>
User Enumeration	מידע לתוקף	"שם משתמש או סיסמה שגויים" (לא מה שגוי)

פרק 5: OpenRouteService – חישוב מסלול

אופניים

5.1 API endpoint

```
GET https://api.openrouteservice.org/v2/directions/cycling-regular/geojson
Headers: Authorization: {ORS_API_KEY}
Body: { "coordinates": [[fromLng, fromLat], [toLng, toLat]] }

Response:
{
  "features": [{
    "geometry": { "type": "LineString", "coordinates": [[lng,lat], ...] },
    "properties": { "summary": { "distance": 14800, "duration": 3480 } }
  }]
}
```

5.2 Fallback כאשר ORS לא זמין

```
if (!ORS_API_KEY || ORS_API_KEY.length < 20) {
  // Haversine קו ישר + חישוב מרחק
  return { distanceKm, durationMin: distanceKm/15*60, geometry: lineString, isFallback: true }
}
```

פרק 6: Leaflet ו SSR

6.1 למה Leaflet דורש Client Component?

1. Leaflet: **window/document** צריך DOM – לא קיים בשרת

2. **Browser APIs:** Canvas, WebGL, requestAnimationFrame

Event Listeners: useEffect, DOM events .3**הפתרון:**

```
// MermaidWrapper.tsx
'use client';
const EmergencyMap = dynamic(() => import('./EmergencyMap'), { ssr: false });
```

6.2 בעיית StrictMode

```
Mount #1 → Leaflet → _leaflet_id ל div נוסף
Unmount → map.remove()
Mount #2 → if (_leaflet_id) return; ← מדלג!
```

6.3 בעיית [lng, lat] vs [lat, lng]

```
GeoJSON/MongoDB/ORS: [longitude, latitude] lng ראשון!
Leaflet:               [latitude, longitude] lat ראשון!

// תל אביב:
MongoDB: { coordinates: [34.7818, 32.0853] } // [lng, lat]
Leaflet: L.marker([32.0853, 34.7818])         // [lat, lng]

// route polyline:
coords.map(c => [c[1], c[0]]) // [lng,lat] → [lat,lng]
```

פרק 7: Gamification**7.1 חישוב ניקוד**

```
ניקוד בסיסי: 10+ נקודות לכל תגובה
בנוסף מהירות: 10+ אם מגיע תוך 5 דקות
אם מגיע תוך 10 דקות +5
```

LoRa (תורם לרשת) LoRa: למי שמסדר +3 LoRa: בונוס
בונוס סוללה: 2+ אם סוללה מעל 80%

7.2 ממוצע מצטבר ב-SQL

```
avg_response_s = (avg_response_s * responses + new_value) / (responses + 1)
-- ללא שמירת כל הנתונים Incremental Mean - O(1) נוסחת
```

7.3 Badge "מציל חיים"

```
badges = CASE
  WHEN total_points + $1 >= 50 AND NOT ('מציל חיים' = ANY(badges))
    THEN array_append(badges, 'מציל חיים')
  ELSE badges
END
-- array_append: הוספת איבר למערך PostgreSQL
-- ANY(badges): בדיקה האם ערך קיים במערך
```

פרק 8: 10 שאלות ריאיון + תשובות

שאלה 1: "למה בחרת ב-MongoDB לנתוני מיקום?"

תשובה:

MongoDB תומך ב-GeoJSON 2dsphere index מובנה. שאילתת "מצא כל המכשירים בטווח X ק"מ" היא שורה אחת:

```
db.device_telemetry.find({
  location: { $geoWithin: { $centerSphere: [[lng, lat], radiusKm/6378.1] }}
});
```

ב-PostgreSQL ללא PostGIS extension זה מצריך Haversine formula מורכבת. בנוסף, אירועי
חירום עם arrays מקוננים של מתנדבים = מתאים ל-Document DB, לא ל-SQL.

שאלה 2: "מה ההבדל בין Access Token ו-Refresh Token?"

תשובה:

Refresh Token	Access Token	
7 ימים	15 דקות	תוקף
HttpOnly Cookie-ב	Memory-ב	שמור
Cookie אוטומטי	Authorization header	נשלח ב
כן (jti + revoked flag)	לא	מאוחסן ב-DB?
כן (revoked=TRUE)	לא (יפוג לבד)	ניתן לביטול?

הה Access Token לא נשמר ב-localStorage כי XSS יכול לגנוב אותו. הה Refresh Token הוא JavaScript – HttpOnly לא יכול לגשת אליו.

שאלה 3: "מה זה 2dsphere index ואיך הוא עובד?"

תשובה:

הוא MongoDB index שמבין גיאומטריה ספריתית על כדור הארץ. מאפשר `$geoWithin`, `2dsphere`, `$geoIntersects`, `$geoNear`.

למה `?radiusInRadians`

עובד על ספירה (sphere): `$centerSphere`

```
radians = km / 6378.1
km / 6378.1 = 0.003136 radians : 20
```

חיוני: index מוגדר על שדה GeoJSON: `{ type: "Point", coordinates: [lng, lat] }`

שאלה 4: "למה EmergencyMap חייב להיות Client Component?"

תשובה:

שלוש סיבות:

1. Leaflet קורא ל- `document.createElement('div')` – לא קיים בשרת
2. Canvas API, WebGL, requestAnimationFrame – Browser-only
3. `useEffect`, `useRef`, event listeners – דורשים DOM

הפתרון: `dynamic(() => import('./EmergencyMap'), { ssr: false })` – נטען רק בדפדפן, לא ב-build time.

שאלה 5: "מה הבעיה עם StrictMode ו-Leaflet?"

תשובה:

React StrictMode מריץ כל `useEffect` פעמיים:

```
Mount → Leaflet → _leaflet_id=5 ← div
Unmount → map.remove()
Mount → CRASH → מנוקה div מנסה Leaflet → שוב Mount
```

הפתרון:

```
השני Mount מדלג על
if ((container as any)._leaflet_id) return;
```

שאלה 6: "הסבר את הבאג עם שם מסד הנתונים"

תשובה:

URI של MongoDB Atlas: `mongodb+srv://user:pass@cluster.net/?appName=Cluster0`

כאשר קוראים `client.db()` ללא ארגומנט על URI ללא שם MongoDB – DB מחזיר את DB ברירת המחדל `test`.

הבאג: `seed.ts` כתב ל- `lora_defi`, `mongodb.ts` קרא מ- `test`. תוצאה: 0 מתנדבים!

הפתרון:

```
db = client.db('lora_defi'); // תמיד מפורש!
```

שאלה 7: "מה זה CORS ולמה הגדרת אותו?"

תשובה:

CORS = Cross-Origin Resource Sharing. הדפדפן חוסם בקשות בין domains שונים.

הבקשה מ- `https://my-app.vercel.app` ל- `https://auth.render.com` נחסמת ללא:

```
app.use(cors({
  origin: process.env.FRONTEND_URL,
  credentials: true // cookies!
}));
```

— חובה ל-`refresh token cookie` לעבור ב-`cross-origin request`. `credentials: true`

שאלה 8: "מה זה Connection Pool ולמה הוא חיוני?"

תשובה:

Client פשוט = פתיחת TCP+TLS חדש לכל בקשה (~100-300ms). ב-10 requests בו-זמנית = בעיית ביצועים קשה.

Pool = מאגר 10 חיבורים פתוחים מראש:

```
pool נעשה שימוש → מוחזר ל (0ms!) בקשה → מושאלת חיבור קיים
```

Singleton ב-Node.js: `const pool = new Pool(...)` מחוץ לפונקציה = module cache = כולם מקבלים את אותו pool.

שאלה 9: "הסבר את בעיית הקואורדינטות [lng,lat] vs [lat,lng]"

תשובה:

GeoJSON/MongoDB/ORS: [longitude, latitude] – אורך גיאוגרפי ראשון!
Leaflet: [latitude, longitude] – רוחב גיאוגרפי ראשון!

תל אביב:

MongoDB: `coordinates: [34.7818, 32.0853]` – [lng, lat] –

Leaflet: `L.marker([32.0853, 34.7818])` – [lat, lng] –

Route – `coords.map(c => [c[1], c[0]])` – היפוך!

שגיאה = סמנים מופיעים בים התיכון במקום בישראל!

שאלה 10: "למה השתמשת ב-bcrypt ולא MD5 או SHA256?"

תשובה:

MD5/SHA256 הם hash functions מהירים – המהירות היא חיסרון לסיסמאות!

bcrypt מוסיף:

1. **Salt:** ערך אקראי ייחודי מוסף לכל סיסמה לפני hashing → גם אם שני משתמשים בחרו "1234", ה-

hash שלהם שונה לחלוטין → מגן מ-Rainbow Table attacks

2. **Work Factor:** bcrypt "מאט" את עצמו בכוונה → brute force לוקח שנים

```
// הכנסה:  
const hash = await bcrypt.hash('1234', 10); // 10 = work factor rounds  
  
// בדיקה:  
const isValid = await bcrypt.compare('1234', hash); // true/false
```

נספח: פקודות שימושיות


```
# הפעלה מקומית
cd auth-server && npm run dev      # Express :4000
cd web-app && npm run dev           # Next.js :3000

# Seed הנתונים מסדי
cd web-app && npx tsx scripts/seed.ts

# TypeScript type check (מריץ Render כפי ש)
cd auth-server && npx tsc --noEmit

# build לייצור
cd web-app && npm run build
```

נספח: Environment Variables

```
# web-app/.env.local
DATABASE_URL=postgresql://postgres.XXXXX:PASSWORD@aws-1-ap-northeast-1.pooler.supabase.co
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/?appName=Cluster0
ORS_API_KEY=eyJvcmdiOiI1Z...
NEXT_PUBLIC_AUTH_SERVER_URL=http://localhost:4000

# auth-server/.env
JWT_SECRET=your-long-random-secret
JWT_REFRESH_SECRET=your-different-secret
DATABASE_URL=postgresql://...
FRONTEND_URL=http://localhost:3000
PORT=4000
```

בהצלחה בהגנה!